

Technical Test - Quality Engineer (Automation)

QA Automation Technical Assessment

Objective

Evaluate your ability to design, implement, and execute a functional UI automation test using industry best practices, demonstrating coding standards, maintainability, and reporting capability.

Assessment Overview

You are required to automate a medium-complexity test scenario using **Playwright** or **Selenium** or similar tools (choose the tool you are most comfortable with). The test should interact with a real website, validate key functionalities, and follow a structured framework design.

Sample Website

Use the sample site for this exercise:

👉 <https://opensource-demo.orangehrmlive.com/>

End-to-End Test Scenario

Automate the following **complex functional workflow**:

Scenario: Employee Lifecycle Management

1. Login

- Use valid credentials:
 - Username: Admin
 - Password: admin123
- Verify successful login (e.g. dashboard visibility).

2. Add a New Employee

- Navigate to the **PIM > Add Employee** module.
- Create a new employee using **data-driven input** (e.g. from a JSON or CSV file).
- Include First Name, Last Name, Employee ID, and upload a profile picture.
- Verify the success message or the new record presence.

3. Edit Employee Information

- Search for the newly added employee by Employee ID.

- Update Job Title and Employment Status.
- Verify the changes are successfully reflected.

4. **Validate Employee via API**

- Use the OrangeHRM API (or simulate API with any public test API like [ReqRes](#) | [Free RES](#) [T API for developers](#)) to verify that the employee details exist/updated correctly.
- Cross-check API data against UI data for consistency.

5. **Delete the Employee**

- Delete the same employee from the UI.
- Verify deletion via both UI and API response.

6. **Logout**

- Confirm successful logout and that the session is invalidated.

Framework & Implementation Guidelines

- Use **Playwright** or **Selenium** with your preferred language (**JavaScript/TypeScript, Python, or Java**).
- Implement **Page Object Model (POM)** for maintainability.
- Include **descriptive assertions** with meaningful messages.
- Follow **clean coding practices** (naming conventions, reusable methods, clear folder structure).
- Integrate a **test runner** (e.g. Playwright Test, Pytest, TestNG, or JUnit).
- Generate an **HTML test report** (e.g. Playwright built-in report, Allure, Extent Report, etc.).
- Record **video of the test run** (Playwright supports this natively, or use screen recorder if using Selenium).
- Add a **README.md** file describing:
 - Setup instructions
 - Framework structure
 - How to run the test
 - Dependencies used

Deliverables

Please upload your completed solution to your **GitHub repository** and share the public link with us.

Your repository should include:

- Source code with test scripts
 - Configuration and dependency files (e.g. `package.json`, `pom.xml`, `requirements.txt`)
 - HTML report of test execution
 - Test run video (can be in repo or shared via drive link)
 - README.md with clear execution steps
-

Submission

- **Timeline:** 3 days from receiving this assignment
- **Deliverable:** GitHub link (public or shared access)
- **Github Repo:** share the repo with below ids -
 -
 -